Numpy Basics:

Author: Nafees

Numerical Python, or "Numpy" for short, is a foundational package on which many of the most common data science packages are built. Numpy provides us with high performance multi-dimensional arrays which we can use as vectors or matrices.

How to create Rank 1 numpy arrays:

In [1]:

```
import numpy as np

an_array = np.array([3, 33, 333])  # Create a rank 1 array

print(type(an_array))              # The type of an ndarray is: "<class 'numpy.ndarra
y'>"
```

```
<class 'numpy.ndarray'>
```

In [2]:

```
# test the shape of the array we just created, it should have just one dimension (Rank
 1)
print(an_array.shape)
```

```
(3,)
```

In [3]:

```
# because this is a 1-rank array, we need only one index to accesss each element
print(an_array[0], an_array[1], an_array[2])
```

```
3 33 333
```

In [4]:

```
an_array[0] =888            # ndarrays are mutable, here we change an element of the ar
ray

print(an_array)
```

```
[888  33 333]
```

How to create a Rank 2 numpy array:

In [5]:

```
another = np.array([[11,12,13],[21,22,23]])   # Create a rank 2 array

print(another)  # print the array

print("The shape is 2 rows, 3 columns: ", another.shape)  # rows x columns

print("Accessing elements [0,0], [0,1], and [1,0] of the ndarray: ", another[0, 0], ",
 ",another[0, 1],", ", another[1, 0])
```

```
[[11 12 13]
 [21 22 23]]
The shape is 2 rows, 3 columns:  (2, 3)
Accessing elements [0,0], [0,1], and [1,0] of the ndarray:  11 ,  12 ,  21
```

There are many way to create numpy arrays:

Here we create a number of different size arrays with different shapes and different pre-filled values. numpy has a number of built in methods which help us quickly and easily create multidimensional arrays.

In [6]:

```
import numpy as np

# create a 2x2 array of zeros
ex1 = np.zeros((2,2))
print(ex1)
a = np.random.random(5)
print(a)
```

```
[[0. 0.]
 [0. 0.]]
[0.14955414 0.00788748 0.09938803 0.18692904 0.23906777]
```

In [7]:

```
# create a 2x2 array filled with 9.0
ex2 = np.full((2,2), 9.0)
print(ex2)
```

```
[[9. 9.]
 [9. 9.]]
```

In [8]:

```
# create a 2x2 matrix with the diagonal 1s and the others 0
ex3 = np.eye(2,2)
print(ex3)
```

```
[[1. 0.]
 [0. 1.]]
```

In [9]:

```
# create an array of ones
ex4 = np.ones((1,2))
print(ex4)
```

```
[[1. 1.]]
```

In [10]:

```
# notice that the above ndarray (ex4) is actually rank 2, it is a 2x1 array
print(ex4.shape)

# which means we need to use two indexes to access an element
print()
print(ex4[0,1])
```

```
(1, 2)

1.0
```

In [11]:

```
# create an array of random floats between 0 and 1
ex5 = np.random.random((2,2))
print(ex5)
```

```
[[0.69485247 0.88718579]
 [0.86483531 0.3260886 ]]
```

Array Indexing

Slice indexing:

In [12]:

```
import numpy as np

# Rank 2 array of shape (3, 4)
an_array = np.array([[11,12,13,14], [21,22,23,24], [31,32,33,34]])
print(an_array)
```

```
[[11 12 13 14]
 [21 22 23 24]
 [31 32 33 34]]
```

Use array slicing to get a subarray consisting of the first 2 rows x 2 columns.

In [13]:

```
a_slice = an_array[:2, 1:3]
print(a_slice)
```

```
[[12 13]
 [22 23]]
```

When you modify a slice, you actually modify the underlying array.

In [14]:

```
print("Before:", an_array[0, 1])   #inspect the element at 0, 1
a_slice[0, 0] = 1000    # a_slice[0, 0] is the same piece of data as an_array[0, 1]
print("After:", an_array[0, 1])
```

```
Before: 12
After: 1000
```

Use both integer indexing & slice indexing

We can use combinations of integer indexing and slice indexing to create different shaped matrices.

In [15]:

```
# Create a Rank 2 array of shape (3, 4)
an_array = np.array([[11,12,13,14], [21,22,23,24], [31,32,33,34]])
print(an_array)
```

```
[[11 12 13 14]
 [21 22 23 24]
 [31 32 33 34]]
```

In [16]:

```
# Using both integer indexing & slicing generates an array of lower rank
row_rank1 = an_array[1, :]    # Rank 1 view

print(row_rank1, row_rank1.shape)  # notice only a single []
```

```
[21 22 23 24] (4,)
```

In [17]:

```
# Slicing alone: generates an array of the same rank as the an_array
row_rank2 = an_array[1:2, :]  # Rank 2 view

print(row_rank2, row_rank2.shape)   # Notice the [[ ]]
```

```
[[21 22 23 24]] (1, 4)
```

In [18]:

```
#We can do the same thing for columns of an array:

print()
col_rank1 = an_array[:, 1]
col_rank2 = an_array[:, 1:2]

print(col_rank1, col_rank1.shape)  # Rank 1
print()
print(col_rank2, col_rank2.shape)  # Rank 2
```

```
[12 22 32] (3,)

[[12]
 [22]
 [32]] (3, 1)
```

Array Indexing for changing elements:

Sometimes it's useful to use an array of indexes to access or change elements.

In [19]:

```
# Create a new array
an_array = np.array([[11,12,13], [21,22,23], [31,32,33], [41,42,43]])

print('Original Array:')
print(an_array)
```

```
Original Array:
[[11 12 13]
 [21 22 23]
 [31 32 33]
 [41 42 43]]
```

In [20]:

```
# Create an array of indices
col_indices = np.array([0, 1, 2, 0])
print('\nCol indices picked : ', col_indices)

row_indices = np.arange(4)
print('\nRows indices picked : ', row_indices)
```

```
Col indices picked :  [0 1 2 0]

Rows indices picked :  [0 1 2 3]
```

In [21]:

```
# Examine the pairings of row_indices and col_indices.  These are the elements we'll ch
ange next.
for row,col in zip(row_indices,col_indices):
    print(row, ", ",col)
```

```
0 ,  0
1 ,  1
2 ,  2
3 ,  0
```

In [22]:

```
# Select one element from each row
print('Values in the array at those indices: ',an_array[row_indices, col_indices])
```

```
Values in the array at those indices:  [11 22 33 41]
```

In [23]:

```
# Change one element from each row using the indices selected
an_array[row_indices, col_indices] += 100000

print('\nChanged Array:')
print(an_array)
```

```
Changed Array:
[[100011     12     13]
 [    21 100022     23]
 [    31     32 100033]
 [100041     42     43]]
```

Boolean Indexing

Array Indexing for changing elements:

In [24]:

```
# create a 3x2 array
an_array = np.array([[11,12], [21, 22], [31, 32]])
print(an_array)
```

```
[[11 12]
 [21 22]
 [31 32]]
```

In [25]:

```
# create a filter which will be boolean values for whether each element meets this cond
ition
filter = (an_array > 15)
print(filter)
```

```
[[False False]
 [ True  True]
 [ True  True]]
```

Notice that the filter is a same size ndarray as an_array which is filled with True for each element whose corresponding element in an_array which is greater than 15 and False for those elements whose value is less than 15.

In [26]:

```
# we can now select just those elements which meet that criteria
print(an_array[filter])
```

```
[21 22 31 32]
```

In [27]:

```
# For short, we could have just used the approach below without the need for the separa
te filter array.

an_array[(an_array > 10) & (an_array <18)]
```

Out[27]:

```
array([11, 12])
```

What is particularly useful is that we can actually change elements in the array applying a similar logical filter. Let's add 100 to all the even values.

In [28]:

```
an_array[(an_array > 10) & (an_array <18)] +=100
print(an_array)
```

```
[[111 112]
 [ 21  22]
 [ 31  32]]
```

Datatypes and Array Operations

Datatypes:

In [29]:

```
ex1 = np.array([11, 12]) # Python assigns the  data type
print(ex1.dtype)
```

```
int32
```

In [30]:

```
ex2 = np.array([11.0, 12.0]) # Python assigns the  data type
print(ex2.dtype)
```

```
float64
```

In [31]:

```
ex3 = np.array([11, 21], dtype=np.int64) #You can also tell Python the  data type
print(ex3.dtype)
```

```
int64
```

In [32]:

```
# you can use this to force floats into integers (using floor function)
ex4 = np.array([11.1,12.7], dtype=np.int64)
print(ex4.dtype)
print()
print(ex4)
```

```
int64

[11 12]
```

In [33]:

```
# you can use this to force integers into floats if you anticipate
# the values may change to floats later
ex5 = np.array([11, 21], dtype=np.float64)
print(ex5.dtype)
print()
print(ex5)
```

```
float64

[11. 21.]
```

Arithmetic Array Operations:

In [34]:

```
x = np.array([[111,112],[121,122]], dtype=np.int)
y = np.array([[211.1,212.1],[221.1,222.1]], dtype=np.float64)

print(x)
print()
print(y)
```

```
[[111 112]
 [121 122]]

[[211.1 212.1]
 [221.1 222.1]]
```

In [35]:

```
# add
print(x + y)         # The plus sign works
print()
print(np.add(x, y))  # so does the numpy function "add"
```

```
[[322.1 324.1]
 [342.1 344.1]]

[[322.1 324.1]
 [342.1 344.1]]
```

In [36]:

```
# subtract
print(x - y)
print()
print(np.subtract(x, y))
```

```
[[-100.1 -100.1]
 [-100.1 -100.1]]

[[-100.1 -100.1]
 [-100.1 -100.1]]
```

In [37]:

```
# multiply
print(x * y)
print()
print(np.multiply(x, y))
```

```
[[23432.1 23755.2]
 [26753.1 27096.2]]

[[23432.1 23755.2]
 [26753.1 27096.2]]
```

In [38]:

```
# divide
print(x / y)
print()
print(np.divide(x, y))
```

```
[[0.52581715 0.52805281]
 [0.54726368 0.54930212]]

[[0.52581715 0.52805281]
 [0.54726368 0.54930212]]
```

In [39]:

```
# square root
print(np.sqrt(x))
```

```
[[10.53565375 10.58300524]
 [11.         11.04536102]]
```

In [40]:

```
# exponent (e ** x)
print(np.exp(x))
```

```
[[1.60948707e+48 4.37503945e+48]
 [3.54513118e+52 9.63666567e+52]]
```

Statistical Methods, Sorting, and Set Operations:

Basic Statistical Operations:

In [41]:

```
# setup a random 2 x 4 matrix
arr = 10 * np.random.randn(2,5)
print(arr)
```

```
[[ 15.23149901 -21.21691971  -6.09830855   0.18912464   4.78903397]
 [ -6.24456545  -0.55444075   5.29860463 -10.80444585   1.9793873 ]]
```

In [42]:

```
# compute the mean for all elements
print(arr.mean())
```

```
-1.7431030757364945
```

In [43]:

```
# compute the means by row
print(arr.mean(axis = 1))
```

```
[-1.42111413 -2.06509202]
```

In [44]:

```
# compute the means by column
print(arr.mean(axis = 0))
```

```
[  4.49346678 -10.88568023  -0.39985196  -5.3076606    3.38421063]
```

In [45]:

```
# sum all the elements
print(arr.sum())
```

```
-17.431030757364944
```

In [46]:

```
# compute the medians
print(np.median(arr, axis = 1))
```

```
[ 0.18912464 -0.55444075]
```

Sorting:

In [47]:

```
# create a 10 element array of randoms
unsorted = np.random.randn(10)

print(unsorted)
```

```
[ 0.36259812  0.52395022 -0.60510403 -1.48657782 -1.66278174 -2.35232075
 -0.38482051  0.23293061  0.22531135  0.18877447]
```

In [48]:

```
# create copy and sort
sorted = np.array(unsorted)
sorted.sort()

print(sorted)
print()
print(unsorted)
```

```
[-2.35232075 -1.66278174 -1.48657782 -0.60510403 -0.38482051  0.18877447
  0.22531135  0.23293061  0.36259812  0.52395022]

[ 0.36259812  0.52395022 -0.60510403 -1.48657782 -1.66278174 -2.35232075
 -0.38482051  0.23293061  0.22531135  0.18877447]
```

In [49]:

```
# inplace sorting
unsorted.sort()

print(unsorted)
```

```
[-2.35232075 -1.66278174 -1.48657782 -0.60510403 -0.38482051  0.18877447
  0.22531135  0.23293061  0.36259812  0.52395022]
```

Finding Unique elements:

In [50]:

```
array = np.array([1,2,1,4,2,1,4,2])

print(np.unique(array))
```

```
[1 2 4]
```

Set Operations with np.array data type:

In [51]:

```
s1 = np.array(['desk','chair','bulb'])
s2 = np.array(['lamp','bulb','chair'])
print(s1, s2)
```

```
['desk' 'chair' 'bulb'] ['lamp' 'bulb' 'chair']
```

In [52]:

```
print( np.intersect1d(s1, s2) )
```

```
['bulb' 'chair']
```

In [53]:

```
print( np.union1d(s1, s2) )
```

```
['bulb' 'chair' 'desk' 'lamp']
```

In [54]:

```
print( np.setdiff1d(s1, s2) )# elements in s1 that are not in s2
```

```
['desk']
```

In [55]:

```
print( np.in1d(s1, s2) )#which element of s1 is also in s2
```

```
[False  True  True]
```

Broadcasting:

In [56]:

```
import numpy as np

start = np.zeros((4,3))
print(start)
```

```
[[0. 0. 0.]
 [0. 0. 0.]
 [0. 0. 0.]
 [0. 0. 0.]]
```

In [57]:

```
# create a rank 1 ndarray with 3 values
add_rows = np.array([1, 0, 2])
print(add_rows)
```

```
[1 0 2]
```

In [58]:

```
y = start + add_rows  # add to each row of 'start' using broadcasting
print(y)
```

```
[[1. 0. 2.]
 [1. 0. 2.]
 [1. 0. 2.]
 [1. 0. 2.]]
```

In [59]:

```
# create an ndarray which is 4 x 1 to broadcast across columns
add_cols = np.array([[0,1,2,3]])
add_cols = add_cols.T

print(add_cols)
```

```
[[0]
 [1]
 [2]
 [3]]
```

In [60]:

```
# add to each column of 'start' using broadcasting
y += add_cols
print(y)
```

```
[[1. 0. 2.]
 [2. 1. 3.]
 [3. 2. 4.]
 [4. 3. 5.]]
```

In [61]:

```
# this will just broadcast in both dimensions
add_scalar = np.array([1,2,3])
print(start+add_scalar)
```

```
[[1. 2. 3.]
 [1. 2. 3.]
 [1. 2. 3.]
 [1. 2. 3.]]
```

Example from the slides:

In [62]:

```
# create our 3x4 matrix
arrA = np.array([[1,2,3,4],[5,6,7,8],[9,10,11,12]])
print(arrA)
```

```
[[ 1  2  3  4]
 [ 5  6  7  8]
 [ 9 10 11 12]]
```

In [63]:

```
# create our 4x1 array
arrB = [0,1,0,2]
print(arrB)
```

```
[0, 1, 0, 2]
```

In [64]:

```
# add the two together using broadcasting
print(arrA + arrB)
```

```
[[ 1  3  3  6]
 [ 5  7  7 10]
 [ 9 11 11 14]]
```

Speedtest: ndarrays vs lists

First setup paramaters for the speed test. We'll be testing time to sum elements in an ndarray versus a list.

In [65]:

```
from numpy import arange
from timeit import Timer

size    = 1000000
timeits = 1000
```

In [66]:

```
# create the ndarray with values 0,1,2...,size-1
nd_array = arange(size)
print( type(nd_array) )
```

```
<class 'numpy.ndarray'>
```

In [67]:

```
# timer expects the operation as a parameter,
# here we pass nd_array.sum()
timer_numpy = Timer("nd_array.sum()", "from __main__ import nd_array")

print("Time taken by numpy ndarray: %f seconds" %
      (timer_numpy.timeit(timeits)/timeits))
```

```
Time taken by numpy ndarray: 0.000421 seconds
```

In [68]:

```
# create the list with values 0,1,2...,size-1
a_list = list(range(size))
print (type(a_list) )
```

```
<class 'list'>
```

In [69]:

```
# timer expects the operation as a parameter, here we pass sum(a_list)
timer_list = Timer("sum(a_list)", "from __main__ import a_list")

print("Time taken by list:  %f seconds" %
      (timer_list.timeit(timeits)/timeits))
```

```
Time taken by list:  0.023964 seconds
```

Read or Write to Disk:

Binary Format:

In [70]:

```
x = np.array([ 23.23, 24.24] )
```

In [71]:

```
np.save('an_array', x)
```

In [72]:

```
np.load('an_array.npy')
```

Out[72]:

```
array([23.23, 24.24])
```

Text Format:

In [73]:

```
np.savetxt('array.txt', X=x, delimiter=',')
```

In [74]:

```
!cat array.txt
```

```
2.323000000000000043e+01
2.423999999999999844e+01
```

In [75]:

```
np.loadtxt('array.txt', delimiter=',')
```

Out[75]:

```
array([23.23, 24.24])
```

Additional Common ndarray Operations

Dot Product on Matrices and Inner Product on Vectors:

In [76]:

```
# determine the dot product of two matrices
x2d = np.array([[1,1],[1,1]])
y2d = np.array([[2,2],[2,2]])

print(x2d.dot(y2d))
print()
print(np.dot(x2d, y2d))
```

```
[[4 4]
 [4 4]]

[[4 4]
 [4 4]]
```

In [77]:

```
# determine the inner product of two vectors
a1d = np.array([9 , 9 ])
b1d = np.array([10, 10])

print(a1d.dot(b1d))
print()
print(np.dot(a1d, b1d))
```

```
180

180
```

In [78]:

```
# dot produce on an array and vector
print(x2d.dot(a1d))
print()
print(np.dot(x2d, a1d))
```

```
[18 18]

[18 18]
```

Sum:

In [79]:

```
# sum elements in the array
ex1 = np.array([[11,12],[21,22]])

print(ex1.sum(axis = 0))          # add all members
```

```
[32 34]
```

In [80]:

```
print(np.sum(ex1, axis=0))  # columnwise sum
```

```
[32 34]
```

In [81]:

```
print(np.sum(ex1, axis=1))  # rowwise sum
```

```
[23 43]
```

Element-wise Functions:

For example, let's compare two arrays values to get the maximum of each.

In [82]:

```
# random array
x = np.random.randn(8)
x
```

Out[82]:

```
array([ 1.74104644,  0.1927069 ,  1.77994502,  0.53431303,  0.12012631,
       -0.3027293 , -0.08020438,  0.09628544])
```

In [83]:

```
# another random array
y = np.random.randn(8)
y
```

Out[83]:

```
array([-0.31050878, -1.05887855,  0.87055958, -0.22207878, -0.35945576,
        0.22591065, -1.62747891,  1.04381541])
```

In [84]:

```
# returns element wise maximum between two arrays

np.maximum(x, y)
```

Out[84]:

```
array([ 1.74104644,  0.1927069 ,  1.77994502,  0.53431303,  0.12012631,
        0.22591065, -0.08020438,  1.04381541])
```

Reshaping array:

In [85]:

```
# grab values from 0 through 19 in an array
arr = np.arange(20)
print(arr)
```

```
[ 0  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19]
```

In [86]:

```
# reshape to be a 4 x 5 matrix
arr.reshape(4,5)
```

Out[86]:

```
array([[ 0,  1,  2,  3,  4],
       [ 5,  6,  7,  8,  9],
       [10, 11, 12, 13, 14],
       [15, 16, 17, 18, 19]])
```

Transpose:

In [87]:

```
# transpose
ex1 = np.array([[11,12],[21,22]])

ex1.T
```

Out[87]:

```
array([[11, 21],
       [12, 22]])
```

Indexing using where():

In [88]:

```
x_1 = np.array([1,2,3,4,5])

y_1 = np.array([11,22,33,44,55])

filter = np.array([True, False, True, False, False])
```

In [89]:

```
out = np.where(filter, y_1, x_1)
print(out)
```

```
[11  2 33  4  5]
```

In [90]:

```
mat = np.random.rand(5,5)
mat
```

Out[90]:

```
array([[0.34112224, 0.27210096, 0.97222392, 0.82631224, 0.80389805],
       [0.7209215 , 0.26400561, 0.88077984, 0.07543192, 0.64411202],
       [0.42077211, 0.55135237, 0.46646193, 0.30318426, 0.68619918],
       [0.10293295, 0.31219932, 0.84350236, 0.9533807 , 0.97383246],
       [0.72086785, 0.30011681, 0.22617193, 0.24039316, 0.88059858]])
```

In [91]:

```
np.where( mat > 0.5, 10 , 2)
```

Out[91]:

```
array([[ 2,  2, 10, 10, 10],
       [10,  2, 10,  2, 10],
       [ 2, 10,  2,  2, 10],
       [ 2,  2, 10, 10, 10],
       [10,  2,  2,  2, 10]])
```

"any" or "all" conditionals:

In [92]:

```
arr_bools = np.array([ True, True, True])
```

In [93]:

```
arr_bools.any()
```

Out[93]:

```
True
```

In [94]:

```
arr_bools.all()
```

Out[94]:

```
True
```

Random Number Generation:

In [95]:

```
Y = np.random.normal(size = (2,5))[0]
print(Y)
```

```
[-0.29053777  0.04697388 -0.89924066 -0.95217471 -1.31479096]
```

In [96]:

```
Z = np.random.randint(low=1,high=50,size=(2,4))
print(Z)
```

```
[[16 15 17 37]
 [37 25 44 41]]
```

In [97]:

```
np.random.permutation(Z) #return a new ordering of elements in Z
```

Out[97]:

```
array([[16, 15, 17, 37],
       [37, 25, 44, 41]])
```

In [98]:

```
np.random.uniform(size=4) #uniform distribution
```

Out[98]:

```
array([0.01003672, 0.88698786, 0.66394578, 0.50199998])
```

In [99]:

```
np.random.normal(size=4) #normal distribution
```

Out[99]:

```
array([ 2.40820301, -0.99052321, -0.00797459,  1.20324499])
```

Merging data sets:

In [100]:

```
K = np.random.randint(low=2,high=50,size=(2,1))
print(K)

print()
M = np.random.randint(low=2,high=50,size=(2,1))
print(M)
```

```
[[35]
 [ 4]]

[[12]
 [27]]
```

In [101]:

```
np.vstack((K,M))
```

Out[101]:

```
array([[35],
       [ 4],
       [12],
       [27]])
```

In [102]:

```
np.hstack((K,M))
```

Out[102]:

```
array([[35, 12],
       [ 4, 27]])
```

In [103]:

```
np.concatenate([K, M], axis = 0)
```

Out[103]:

```
array([[35],
       [ 4],
       [12],
       [27]])
```

In [ ]:

```

```